AI-Based Solution for Analysing, Benchmarking and Quality Monitoring of Cybersecurity Audit Reports

DEMONSTRATION AND VISUALISATION MANUAL

Objective 1: Standardisation of Cybersecurity Audit Report Format

Contents

1. Introduction
2. Demonstration Plan
3. Demonstration Environment and Preparation
4. The Process at a Glance
5. Demonstration Scenarios (Step by Step)
6. Stage-by-Stage Visualisation
7. Measured Results from the Sample Reports
8. Interpreting and Verifying the Output
9. Evaluation Checklist
10. Limitations to Declare During the Demonstration
11. Contingency and Troubleshooting
Appendix A: Presenter Script Sheet
Appendix B: Glossary

1. Introduction

1.1 Purpose

This manual guides a presenter through a live demonstration of the Report-to-JSON pipeline and helps evaluators interpret what they see. It shows how an audit report moves from a PDF or Word file to a standardised JSON record, what each intermediate stage produces, and how to check the result against the original report.

1.2 Audience

Role	Use of this manual
Presenter / operator	Runs the demonstration using the scenarios in Section 5 and the script sheet in Appendix A
Evaluator / reviewer	Uses Sections 6 to 9 to judge the output and fills in Appendix B
Project management	Uses Sections 2, 7 and 10 to plan the session and decide what to show

1.3 What the Demonstration Proves, and What It Does Not

Shown in this demonstration	Not shown (see Section 10)
A report converted into the standard JSON structure	Scanned-report OCR; TXT, XLSX and log inputs
Intermediate Markdown transcript and extracted images	Per-field page and paragraph references
Linking of findings to screenshots by token	Human review, edit and approval workflow
Speed of the rule-based engine on six sample reports	Benchmarking, dashboard and agency scoring (Objectives 2 to 5)
How to verify output against the source report	AI-model runs, unless the models are installed on the demo machine

1.4 Conventions

Convention	Meaning
Monospaced text	A command, file name, path or field name
R1 to R6	The six sample reports in VAPT-API/ (1-report.pdf to 6-report.pdf)
Green / amber / red	In figures: matches the source / partly correct / wrong or not in the source
<host>, <Agency>	Real names removed from excerpts in this manual

2. Demonstration Plan

2.1 Objectives of the Demonstration

1. Show that the pipeline converts an unstructured audit report into the Master Schema format end to end.
2. Make every intermediate stage visible, so evaluators can see where each value comes from.
3. Give an honest account of accuracy: what is extracted correctly, what is estimated, and what is missing.
4. Collect structured evaluator feedback.

2.2 Roles

Role	Responsibility
Presenter	Explains each step and the meaning of what is on screen
Operator	Runs commands and the GUI; may be the presenter
Evaluator(s)	Watch, ask questions, check values against the source, complete Appendix B
Technical support	Standby for set-up faults; holds the pre-generated artefacts (Appendix C)

2.3 Suggested Schedule

Time	Part	Section
0 to 3 min	Introduction and the decision-support statement	1.3
3 to 8 min	Scenario A: quick command-line run on a 2-page sample	5.1
8 to 16 min	Scenario B: full run on an 18-page report in the desktop application	5.2
16 to 21 min	Scenario C: look inside the intermediate stages	5.3 and 6
21 to 25 min	Verification of the output against the source	8
25 to 30 min	Questions and scoring	9, Appendix B

2.4 Success Criteria

- Each demonstrated run ends with status success and the three output artefacts present.
- Evaluators can trace at least one finding from the JSON back to the source page and its screenshot.
- Limitations in Section 10 are stated openly before evaluators are asked to score.

3. Demonstration Environment and Preparation

3.1 Equipment

Item	Requirement
Machine	Inside the approved secure environment; display or projector for the audience
Rule-based demonstration	Python environment with the packages in requirements.txt (CPU only is enough)
AI-engine demonstration (optional)	NVIDIA GPU with 8 GB VRAM, CUDA 12 or later, and the Nemotron-Mini or Phi-3.5 Vision model already stored locally
Security gate demonstration (optional)	The YARA scanner module and rules, which are not part of the repository; see Section 5.5

3.2 Sample Data

ID	File	Pages	Size	Suitability for a live demonstration
R1	VAPT-API/1-report.pdf	18	1.2 MB	Recommended. Two findings; extraction matches the source for the main fields
R2	VAPT-API/2-report.pdf	29	1.7 MB	Recommended. Seven findings; severities match the source
R3	VAPT-API/3-report.pdf	27	1.1 MB	Not recommended for the rule-based engine; entries are section headings (Section 7)
R4	VAPT-API/4-report.pdf	17	1.1 MB	As R3
R5	VAPT-API/5-report.pdf	18	1.9 MB	As R3; the source holds 16 severity ratings but only 6 entries are produced
R6	VAPT-API/6-report.pdf	28	1.3 MB	As R3
S	outputs/sample_2page.pdf	2	0.2 MB	Recommended for the quick run; cover and document-control pages

3.3 Preparation Checklist

#	Task
1	Activate the virtual environment and confirm the application starts (python -u gui_app.py)
2	Run Scenario A and Scenario B once as a rehearsal; note the time taken
3	Copy the rehearsal outputs to a backup folder (Appendix C) in case the live run fails
4	If an AI engine will be shown: run python download_models.py --status and confirm the model is present; run one report beforehand

#	Task
5	If the security gate will be shown: confirm the scanner module and rules are installed (python pipeline_integration.py <file>)
6	Open the source PDF of R1 beside the output folder so that page 10 and page 11 can be shown quickly
7	Clear the demonstration output folder so the audience sees only fresh results
8	Print or share Appendix B with the evaluators

4. The Process at a Glance

Figure 1 shows the path of one report. The left column is what the software does; the right column is what the audience can see at that point. The same six steps structure every scenario in Section 5.

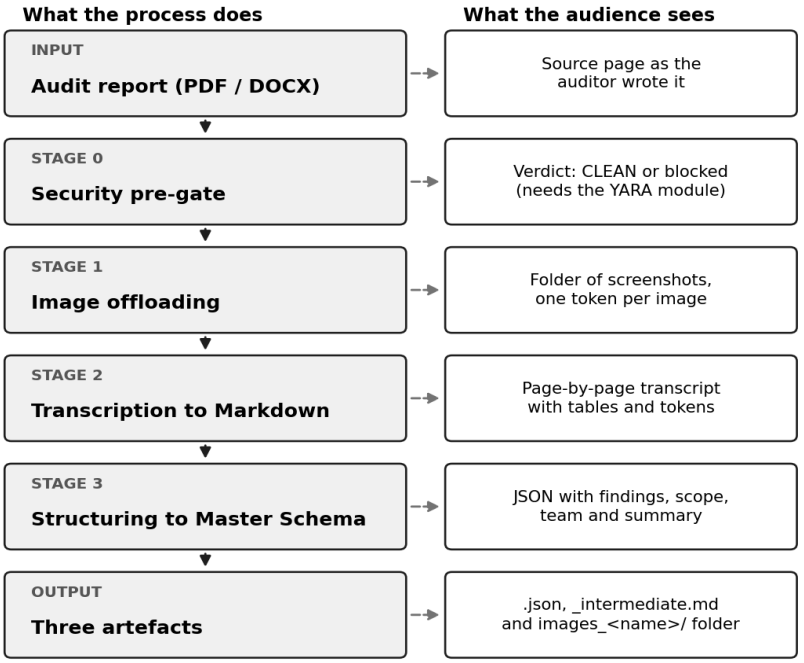


Figure 1: Process steps and what the audience sees at each step

4.1 Choosing an Engine for the Demonstration

Engine	Needs	Use in the demonstration
Rule-Based (Fast Layout Parser)	CPU only	Default for live demonstration: finishes in seconds and repeats exactly
Nemotron-Mini 4B	GPU 8 GB, model on disk	Optional: shows AI structuring against the schema
Phi-3.5 Vision	GPU 8 GB, model on disk	Optional: shows page-image transcription

5. Demonstration Scenarios (Step by Step)

5.1 Scenario A: Quick Run from the Command Line (about 5 minutes)

Goal. Show the complete pipeline on a 2-page sample in under a second.

1. Open a terminal in the project folder.
2. Run the command:

   **python -m vlm_repo.src.pipeline --input outputs/sample_2page.pdf **

   --output demo_out/sample_2page.json --backend rule_based

3. Point to the console messages: Stage 1 (images), Stage 2 (transcript), Stage 3 (JSON), and the line "Pipeline Completed Successfully".
4. List the folder: three artefacts appear (the JSON, the _intermediate.md transcript and the images_sample_2page folder, which is empty because this sample has no images).

5.2 Scenario B: Full Run in the Desktop Application

Goal. Show the graphical workflow on a complete report with images and findings (R1).

1. Start the application: python -u gui_app.py.
2. In card 1, click the drop zone and choose VAPT-API/1-report.pdf (or pick it from Quick Sample).
3. In card 2, choose Fast Layout Parser (Rule-Based). Leave Malware Pre-Gate ticked.
4. In card 3, check that Out Dir is the demonstration folder and Schema is vlm_repo/master_schema.json.
5. Click PROCESS REPORT TO STANDARDIZED JSON. Narrate the progress screen: five stages change from Waiting to Processing to Completed.
6. When the results open, show the summary bar (2 observations; 0 critical, 2 high, 0 medium, 0 low).
7. Walk through the tabs: Standardized JSON (use Search JSON to find "Broken Authentication"), Findings & Observations (two cards), Markdown Transcript (page markers and tables) and Live Logs (7 images extracted).
8. Click Extracted Figures to open the images folder (7 screenshots).

5.3 Scenario C: Look Inside the Stages

Goal. Show that the stages are separate and inspectable.

1. Run only Stages 1 and 2 (no AI call):

   **python -m vlm_repo.src.pipeline --input VAPT-API/1-report.pdf **

   --output demo_out/r1_dry.json --backend rule_based --dry-run

2. Open demo_out/1-report_intermediate.md or the file next to the output, and show the page markers, the table blocks and the line [IMAGE_PAGE_11_FIG_1].
3. Show the images folder and open page_11_fig_1.jpeg.
4. Optionally re-run only Stage 3 from the transcript: add --stage 3 and give the .md file as input. This proves that structuring can be repeated without re-reading the PDF.

5.4 Scenario D: AI Engine

Run only if the model is on the machine and has been tried beforehand. In the desktop application choose NVIDIA Nemotron-Mini 4B (4-Bit) and wait for the status "Ready (Model cached locally on disk)". Process the same report. Afterwards compare _meta.structuring_backend: a value starting with local_vlm_ means the model produced the JSON; rule_based_fallback means the model failed and the rule-based route was used. Do not describe a fallback result as an AI result.

5.5 Scenario E: Security Gate

The gate needs the YARA scanner module and rules, which are not in the repository. If they are installed, run python create_test_samples.py to produce the harmless EICAR test documents, then process one of them. The expected result is a blocked message (Document blocked by security scanner) and no outputs. If the scanner is not installed, do not run this scenario: the application would not perform a scan.

5.6 Scenario F: Web Interface

1. Start the server: python web_app.py (default port 8000).
2. Open http://localhost:8000, select a PDF, choose Fast Layout Parser (Rule-Based) and submit.
3. Show the returned result and the JSON file in outputs/.

The web interface has no login; run it only on an isolated network and never show it to remote viewers.

5.7 Scenario G: Batch of Six Reports (Optional, about 1 minute of run time)

python -m vlm_repo.src.pipeline --input-dir VAPT-API --output-dir demo_out/batch --backend rule_based

This processes all six samples in about 45 seconds of run time on the review machine (Section 7). Use it to discuss throughput, and be ready to explain Figure 8.

6. Stage-by-Stage Visualisation

This section is the reference for the explanation to evaluators. All examples come from R1 (the 18-page report) processed by the rule-based engine.

6.1 From Source to Transcript to JSON

Figure 3 shows the same finding at the three points in its journey. Panel A is how the auditor wrote it, panel B is the intermediate transcript, and panel C is the final record.

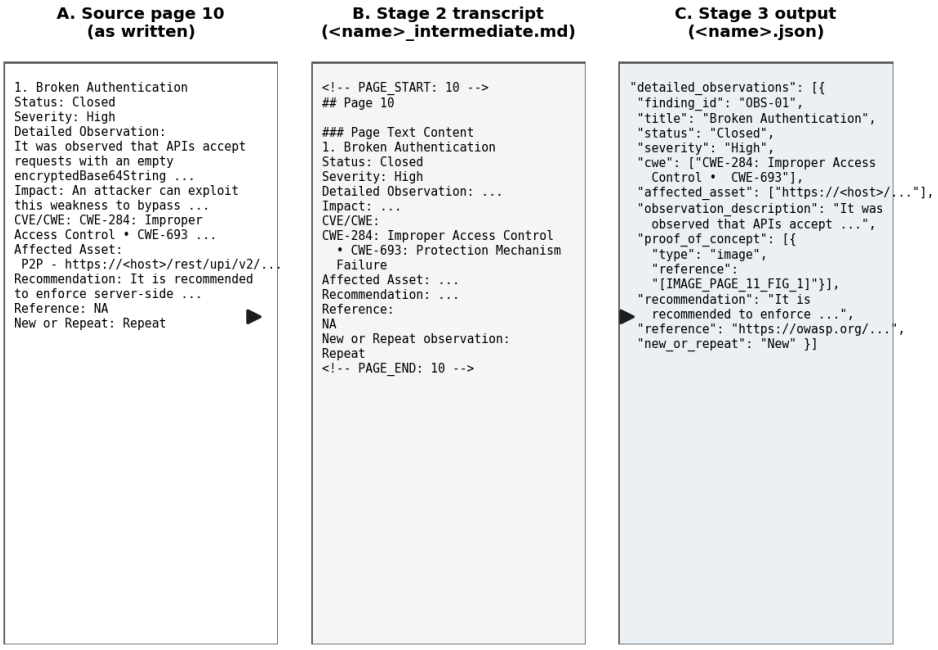


Figure 3: One finding (page 10) at source, transcript and JSON stage

Observe that the transcript keeps the text almost exactly and adds structure (page markers, headings, table blocks). The JSON then selects and labels values. Two values in panel C differ from panel A: new_or_repeat and reference. Section 8 explains why.

6.2 Stage 0: Security Pre-Gate

Item	Detail
Input	The uploaded file, before any parser opens it
Output	A verdict (CLEAN, SUSPICIOUS, MALICIOUS or TEST_SIGNATURE_DETECTED); blocked files stop here
What to show	The Live Logs line "Pre-Gate Verdict". Only meaningful when the scanner module is installed

6.3 Stage 1: Image Offloading

All embedded images are saved to disk and replaced in the text by tokens. For R1 the stage extracted seven images:

Token	File	Size (pixels)	Page
[IMAGE_PAGE_7_FIG_1]	page_7_fig_1.jpeg	1600 x 900	7
[IMAGE_PAGE_11_FIG_1]	page_11_fig_1.jpeg	1041 x 550	11
[IMAGE_PAGE_12_FIG_1]	page_12_fig_1.jpeg	1271 x 595	12
[IMAGE_PAGE_13_FIG_1]	page_13_fig_1.jpeg	1271 x 595	13
[IMAGE_PAGE_14_FIG_1]	page_14_fig_1.jpeg	1702 x 1131	14
[IMAGE_PAGE_16_FIG_1]	page_16_fig_1.jpeg	1273 x 624	16
[IMAGE_PAGE_17_FIG_1]	page_17_fig_1.jpeg	1656 x 766	17

6.4 Stage 2: Transcript

The transcript is a readable Markdown file with a header (pages, tables, characters), one block per page and a block per detected table. Structure to point out:

```
# Document Transcript: 1-report.pdf
*Extracted Pages: 18 | Detected Tables: 16 | Total Chars: 17989*
<!-- PAGE_START: 10 -->
## Page 10
### Page Text Content
1. Broken Authentication
Status: Closed
Severity: High
...
<!-- PAGE_END: 10 -->
```

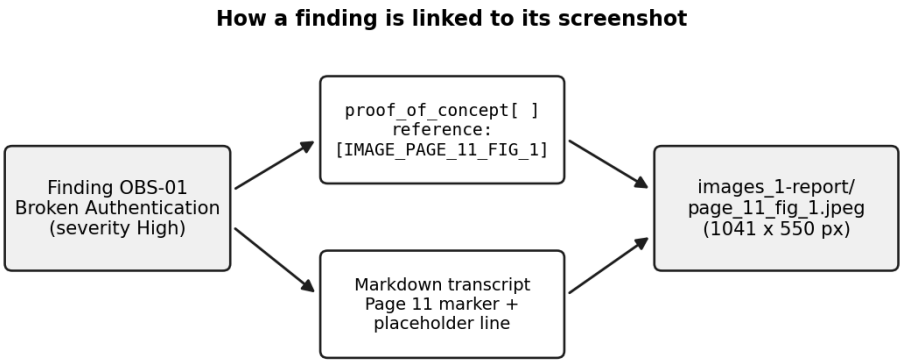
6.5 Stage 3: Structured JSON

The JSON holds the ten Master Schema sections. For R1 the rule-based run produced:

Section	What the demonstration shows
report_meta	Title, type, release date 16.02.2026, audit period 29.07.2025 to 15.02.2026, classification Confidential
document_control	Prepared, reviewed, approved and released by (names masked here)
engagement_scope	Three assets with criticality High, exclusions and credentials
auditing_team	Team members with designation and certifications (masked here)
methodology and tools_used	OWASP API Security Top 10, CERT-In Guidelines, NIST SP 800-115; Postman 11.80.1 and Burp Suite Pro 2026.1

Section	What the demonstration shows
executive_summary	2 findings, 2 high; summary table with one row per finding
detailed_observations	OBS-01 and OBS-02 with description, impact, recommendation and evidence tokens
appendices	Four risk-rating rows; no retest or closure records

6.6 How a Finding Is Linked to Its Evidence



The link is the token text; no coordinates or captions are stored (see Section 8.3).

Figure 4: Linking a finding to its screenshot

7. Measured Results from the Sample Reports

The six sample reports were processed live with the rule-based engine on the review machine (CPU only). Times include Python start-up and will differ on the demonstration machine.

ID	Pages	Tables	Images	Chars	Findings	Time (s)	Assessment of the extracted findings
R1	18	16	7	17,989	2	2.3	Correct: two real findings, both High, as in the source
R2	29	18	16	33,131	7	2.7	Correct titles; severities 1 High, 4 Medium, 2 Low as in the source
R3	27	69	297	79,417	10	12.0	Incorrect: entries are section headings (for example Planning, Discovery); all rated High
R4	17	57	187	64,429	15	8.8	Incorrect: as R3
R5	18	41	180	70,647	6	4.9	Incorrect: source holds 16 severity ratings (6 High, 10 Medium); the entries are headings
R6	28	54	364	83,379	18	13.7	Incorrect: as R3

7.1 What the Results Mean

- Speed is not the issue: all six reports finish in under 14 seconds each.
- The rule-based engine suits reports laid out like R1 and R2 (a numbered finding followed by "Status:" and "Severity:" lines). It does not recognise the layouts of R3 to R6, so it turns headings into findings and gives them a default High rating.
- The large image counts for R3 to R6 (180 to 364) show that many small pieces of page artwork are being extracted as images. Evaluators should not read the number as screenshots of evidence.
- A finding count alone is not a quality measure. Always check titles against the source summary table.

8. Interpreting and Verifying the Output

8.1 Check What Produced the File

Open the JSON and read the _meta block at the end. It states the route used.

_meta.structuring_backend	Meaning for the evaluator
local_vlm_<model>	An AI model produced the JSON. Verify the content
rule_based	The regular-expression parser produced the JSON. Expect estimated values (Section 8.2)
rule_based_fallback	An AI model was selected but failed; the rule-based route was used instead

8.2 Field-by-Field Check of One Finding

Figure compares every value of finding OBS-01 (R1) with the source page. It is the model for the verification step in the demonstration.

Field	Value in JSON	Check against source page 10
title	Broken Authentication	Matches source
status	Closed	Matches source
severity	High	Matches source
observation_description	It was observed that APIs a...	Matches source
impact	An attacker can exploit ...	Matches source
recommendation	It is recommended to enforc...	Matches source
affected_asset	2 URLs	Both URLs present
cwe	CWE-284 ... • CWE-693	Two CWEs merged; 2nd name cut
observation_date	16.02.2026	Release date reused
new_or_repeat	New	Source says Repeat
reference	owasp.org/...api-security	Source says NA
cvss_score	7.2	Not in source; guessed
cvss_vector	CVSS:3.1/AV:N/AC:L/PR:N/...	Not in source; fixed default
severity_rationale	Vulnerability identified ...	Not in source; fixed text
preconditions	Attacker has network access...	Not in source; fixed text
likely_root_cause	Missing server-side authori...	Not in source; fixed text
regulatory_references	OWASP API Top 10; CERT-In	Not in source; fixed list

Green = matches source Amber = partly correct Red = wrong or not in the source

Figure: Source check of finding OBS-01 (R1, rule-based engine)

Reading the figure:

- **Green (7 fields).** Title, status, severity, description, impact, recommendation and both affected URLs match the source.
- **Amber (2 fields).** Two CWE identifiers are joined into one text value and the second name is cut; observation_date repeats the report release date instead of the date of the finding.

- **Red (8 fields).** new_or_repeat says New although the source says Repeat; reference holds a web address although the source says NA; CVSS score, CVSS vector, severity rationale, preconditions, root cause and regulatory references are not in the source at all. They are fixed values built into the rule-based parser. A fixed pattern such as CVSS 7.2 appearing for every High finding is the signal.

8.3 Where the Link to the Source Stops

The evaluator can trace a value by searching for its text in the transcript, because the transcript keeps page markers. The JSON itself does not hold the page number, paragraph or table of each field, and the image link is the token text only. This is a known gap against the traceability requirement and should be stated in the demonstration.

8.4 Engine Comparison on the 2-Page Sample

The repository holds an earlier output of the same sample made with Nemotron-Mini (through an Ollama server, an earlier code version). The table compares it with the live rule-based run for the facts printed on the source cover and document-control pages.

Field	Source says	Rule-based (live run)	Nemotron-Mini (stored)
Report title	API Assessment Report	API Assessment / VAPT Report (wrong)	API Assessment Report (correct)
Report version	1.1	1.0 (wrong)	1.1 (correct)
Audit type	Follow Up Report	Initial Audit Report (wrong)	Follow Up Report (correct)
Prepared by	A named person	Empty	Correct name
Change history	Two entries	None	Two entries (correct)
Release date	16.02.2026	16.02.2026 (correct)	2026-02-16 (correct, ISO form)
Findings	None (cover pages)	None	One placeholder entry with every value NA

The comparison shows the practical difference: the AI route read the cover facts correctly and left absent values as NA, while the rule-based route replaced several cover facts with fixed text. The stored Nemotron-Mini file was produced by an earlier version of the code, so the AI result should be regenerated on the demonstration machine before it is relied upon.

8.5 Evaluator Verification Procedure

1. Open _meta and note the route.
2. Pick one finding. Find its heading in the transcript and compare status, severity, description, impact and recommendation with the JSON.
3. For CVSS, CWE, CVE and reference: look for the value in the transcript. If it is not there, record it as not from the source.
4. Compare the number and titles of findings with the report's own summary table.
5. Open one image through its token and confirm it belongs to the finding.
6. Record results in Appendix B.

9. Evaluation Checklist

The table maps what the demonstration can show to the Objective 1 activities. The "Observed" column holds the result expected from the review run; the evaluator confirms it live.

Objective 1 activity	**Observed**	**Evidence in the demonstration**
Standard schema (JSON)	Met	Master Schema sections visible in every output
PDF and DOCX processing	Met for PDF; DOCX not demonstrated	Scenarios A and B
Image and evidence extraction	Met	Images folder, tokens in transcript and JSON
Table extraction	Met	Table blocks in the transcript (16 for R1)
Field extraction accuracy	Partly met	Section 8.2; R1 and R2 reliable for main fields, estimated values elsewhere; R3 to R6 not reliable (rule-based)
AI-assisted extraction	Not shown unless Scenario D is run	Scenario D and Section 8.4
Text-to-evidence linking	Partly met	Token link only (Figure 4)
Source traceability per field	Not met	No page or paragraph reference in the JSON
Secure upload, hashing, malware workflow	Partly met	Gate needs missing scanner; no hashing
OCR, TXT, XLSX, logs	Not met	Not supported in the interfaces
Decoupled structure-update module	Not shown	Schema is an external file; no versioned update tool

9.1 Suggested Scoring

For each criterion, the evaluator records Y (met), P (partly) or N (not met), with a short comment in Appendix B. Scoring should be done after Section 10 has been presented, so that the score reflects what was shown and what was declared.

10. Limitations to Declare During the Demonstration

These points were found by running the system and reading the code. They should be told to evaluators before scoring.

#	Limitation	Where shown
1	The rule-based engine and the automatic fallback insert fixed values when a field is not found (CVSS, CWE, dates, reference, impact and recommendation wording, new_or_repeat = New). They are not taken from the report	Section 8.2, Figure 6
2	The rule-based engine misreads the layouts of R3 to R6, producing section headings as findings, all rated High	Section 7, Figure 5
3	Report cover facts can be wrong in rule-based output (title, version, audit type, prepared by)	Section 8.4
4	No OCR; only PDF and DOCX are accepted in the interfaces	Section 9
5	No page, paragraph or table reference per field; no confidence value or explanation per field	Section 8.3
6	The security pre-gate needs a scanner module that is not in the repository; without it no scan takes place, yet the application log still reports a pass	Section 5.5
7	No human review, edit or approval step is provided in the tool	Section 9
8	The web interface has no login	Section 5.6
9	AI-engine results were not generated for this manual (no GPU or models on the review machine); stored AI outputs come from an earlier code version	Section 8.4
10	Pipeline version labels differ between the application window (v3.1), the code (3.2.0) and the output files (3.1.0-decoupled in the live runs)	_meta block

11. Contingency and Troubleshooting

Problem during the demonstration	Likely cause	Action
Application will not start	Wrong environment or missing package	Activate the virtual environment; run pip install -r requirements.txt (offline: from the approved package store); fall back to the command line
"Please select a valid PDF or DOCX report first."	No file chosen	Select the file again
Run is slow or stops on an AI engine	Model loading or GPU memory	Switch to the Rule-Based engine; show the pre-generated outputs for the AI result
Status shows "Download required on first run"	Model not on this machine	Do not download in the secure zone; use the Rule-Based engine
Document blocked by security scanner	Scanner verdict MALICIOUS or SUSPICIOUS	Expected for test samples; for a real report, stop and inform security staff
Output has few or wrong findings	Layout not recognised by the rule-based engine	Explain Section 7; use R1 or R2, or show the pre-generated AI output
Page appears blank in the images folder	Report has no embedded images	Expected for the 2-page sample
Evaluator asks to see a value's source page	Not stored in the JSON	Search the value in the transcript, then open the PDF at that page

Appendix A: Presenter Script Sheet

Time	Action	Key message
0:00	Show Section 1.3 table	The tool converts and structures. It does not decide anything about an agency
3:00	Run Scenario A	Whole pipeline in a second on a 2-page sample
8:00	Start Scenario B; narrate the progress screen	Five visible stages; nothing leaves this machine
12:00	Show tabs: JSON, Findings, Transcript, Logs	Same data in four views
16:00	Open transcript and image folder (Scenario C)	Every stage leaves a file that can be inspected
19:00	Search the token [IMAGE_PAGE_11_FIG_1] in JSON, transcript and folder	Finding to screenshot link
21:00	Show Figure 6; check three values against page 10	Some values are exact, some are estimated: always verify
24:00	State the limitations in Section 10	What to improve before the next phase
25:00	Questions; hand out Appendix B	Collect scores and comments

Appendix B: Glossary

Term	Meaning
Intermediate transcript	Markdown version of the report produced at Stage 2
Master Schema	The standard JSON structure for an audit report
Token / placeholder	Text such as [IMAGE_PAGE_11_FIG_1] standing for an extracted image
Rule-based engine	Parser using layout and text patterns; no AI model
Fallback	Automatic switch to the rule-based route when an AI model fails
Pre-gate	Malware scan applied before a file is opened
_meta	Block at the end of the JSON recording which route produced it
CVSS / CWE / CVE	Vulnerability scoring and identification schemes